

Physics 0525: Lab 11

Arduino I

Equipment: DMM, oscilloscope, Arduino, Sparkfun Inventor's Kit, computer with the Arduino IDE (Integrated Development Environment) installed. The lab has a few laptops and you may borrow one of these. If possible, bring your own laptop and install the Arduino IDE from <https://www.arduino.cc/en/Main/Software>.

Reading: Read through this assignment carefully before coming to the lab. The Arduino is programmed in C/C++ language. If you are not familiar with C, read pages 4-5 of the **SparkFun Inventor's Guide**. You will be given the code for the initial assignments.

Preliminaries: The goal of this and following labs, is to introduce you to the concept of digital electronics in a fun and entertaining way using the Arduino Uno. By doing these experiments you will develop knowledge and skills that will help you in future lab courses and in your career. Today, you will learn about digital and analog input/output, and how to read and control external devices with the Arduino.

Arduino is an open-source electronics platform based on an easy-to-use hardware and software that can interact with the physical world. The most popular type of Arduino is the UNO, shown in Figure 1.

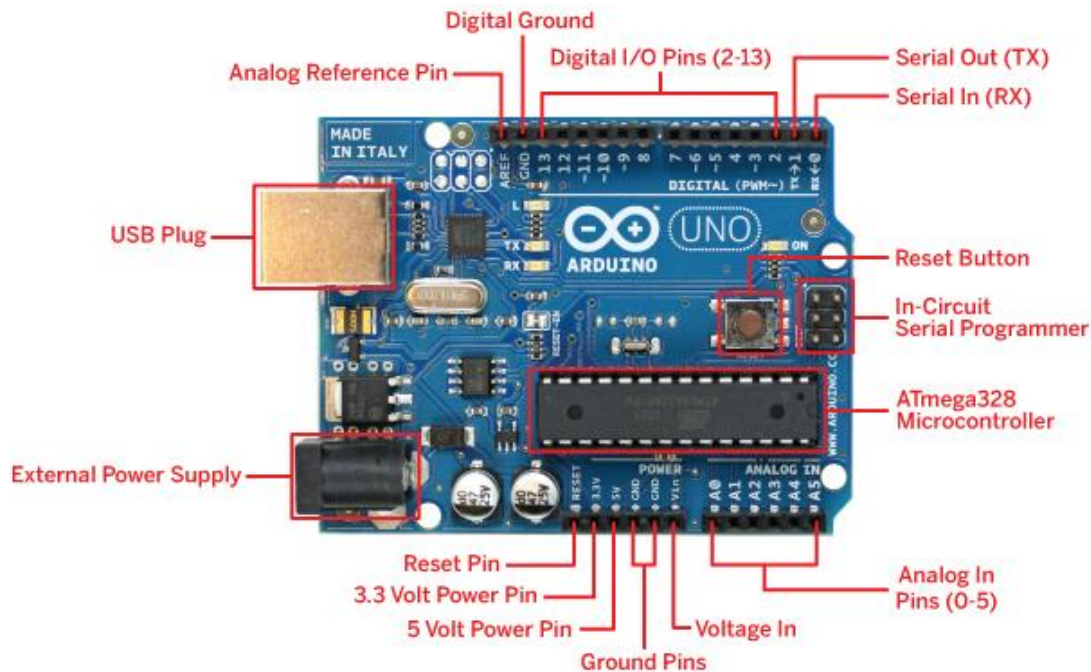


Figure 1: Arduino UNO

The brain of the Arduino UNO is the microcontroller (ATmega328). The microcontroller is a tiny computer that includes a CPU, memory, input/output (I/O) ports, analog-to-digital converters (ADC), and serial interfaces. Its purpose is to control things by interfacing with external sensors and actuators. For instance, you might attach a sensor to measure the temperature and a relay to control the power to a heater. Figure 2 shows some of the things you can attach to an Arduino board.

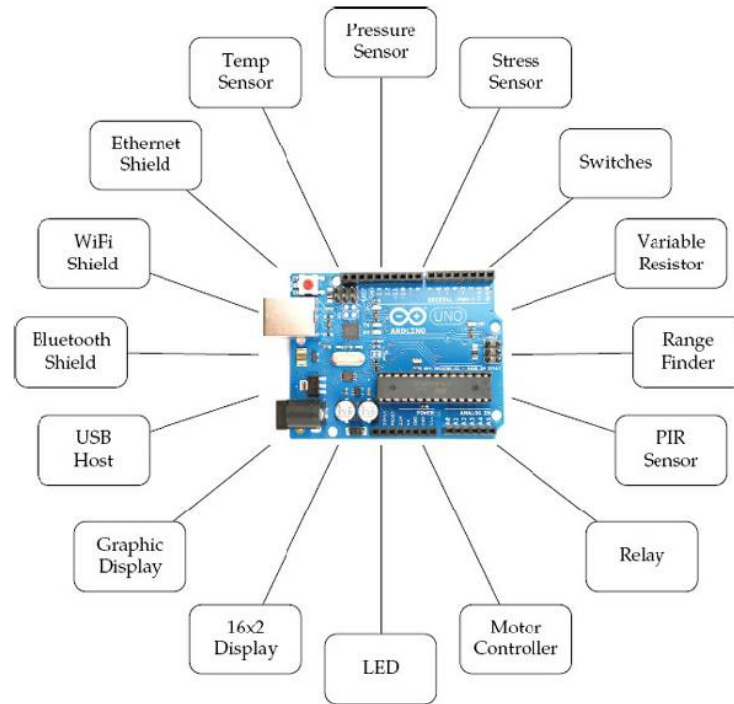


Figure 2. Interfacing with an Arduino (Simon Monk, **Programming Arduino, Next Steps**, McGraw-Hill Education, 2014)

To use an Arduino, you will need a conventional computer (Mac, Windows, Linux). This computer is used to create and upload programs (called sketches) onto the Arduino board. Once uploaded to the Arduino, these programs are retained and can run independently. The Arduino can also communicate with your computer over USB and you can send messages in both directions while the computer is connected.

Anatomy of the Arduino UNO board

The major components of the Arduino UNO are identified on Figure 1. These are described briefly in the list below. There is an appendix at the end of this document that contains technical information and specifications for the UNO for future reference.

- USB port: To connect the Arduino to the host computer in order to power the Arduino, to upload programs (sketches) to the Arduino, or to communicate with the Arduino.
- Power Socket: The Arduino can be powered using a DC power supply or battery (between 7.5 and 12 V) so that it can operate without the host computer. The Arduino only uses about

50 mA, so a small 9 V battery (600 mAh) can power it for 10-12 hours. Since the sketch is stored in non-volatile flash memory, it is not lost when the Arduino is powered down.

- **Reset Pin:** This pin, and the Reset Button, allows you to reboot the Arduino. Setting this pin momentarily to low (connecting it to GND) resets the microcontroller to begin its program from the start. Reset happens automatically when you power-up the Arduino or upload a sketch.
- **3.3 V Power Pin:** Provides 3.3 V power to devices and circuits connected to the Arduino.
- **5 V Power Pin:** Provides 5 V power to Arduino accessories.
- **Ground Pins:** GND (or zero volts) is the voltage to which all other voltages on the board are referenced.
- **Digital I/O Pins (2-13):** Used as either digital inputs or outputs. Digital inputs and output have only two values: high or low (see Table 1). When used as an input, these pins allow the sketch to read the status of a switch, or other device. When used as an output, the sketch controls the value of the pin and can turn an LED, or other device, on or off.
- **Analog Input Pins (A0-A5):** Allows the sketch to measure the voltage of a sensor. Note that the Arduino can only measure positive voltages in the range 0 to 5 V (see Table 1). The input voltage is digitized by the microcontroller using a 10-bit ADC. Although labeled as analog inputs, these pins can also be used as digital inputs or outputs.
- **Analog Output:** The Arduino does not have a true analog output (cannot output a voltage other than 5V). However, some of the digital I/O Pins labeled with a “~” can be used as PWM (pulse width modulation) outputs. PWM is a way of approximating analog output by controlling the duty cycle of a stream of digital pulses. Slower devices respond to the average voltage instead of the individual pulses. The longer the pulse is high, the higher the average voltage.
- **Serial Input/Output (RX/TX):** Digital pins 0 and 1 are dedicated to receive/transmit serial data to/from the USB Plug.

Table 1. Analog and Digital Values

Type	Arduino Range of Values
Analog Input	0-5 V (in 1024 discrete steps)
Digital I/O	High (5 V) or Low (0 V)

Arduino Software Development Environment (IDE)

A screenshot of the Arduino IDE is shown in Figure 3. The sketch displayed is called **Blink**, which repeatedly turns an LED on and off. The programming language is C/C++. Several important characteristics of the C language are:

1. Every statement is terminated by a semicolon “;”
2. Multi-statement blocks are enclosed in curly brackets “{ }”
3. Comments are preceded by “//” or enclosed between “/*” and “*/” delimiters
4. Variables, constants, functions... (all known as identifiers) must be defined before they can be used. Defining an identifier means specifying its name and type (int, double, void...).

Name: _____

Date: _____

A sketch includes the following:

1. Opening comments: These state the purpose of the sketch, provide some information about the circuit that it accompanies, and give the author and date of last modification.
2. Global constants and Variables: These identifiers are known everywhere in the sketch and do not need to be passed as parameters in functions.
3. Setup function: This function is executed only once when the Arduino is powered up, or when you press the reset button. Typically, it is used to configure the sketch by specifying pin assignments and pin modes (input/output).
4. Loop function: This function is executed repeatedly, forever, and is used to input data from sensors and output control commands to actuators.

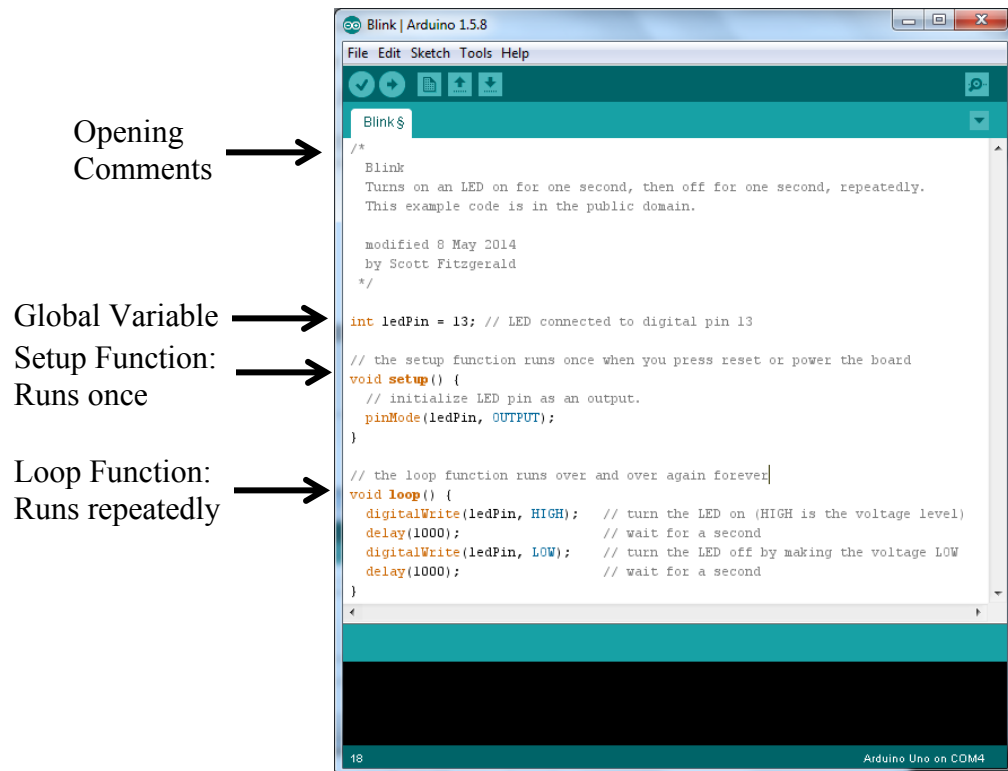


Figure 3. Screenshot of Arduino IDE

Notice that the Arduino IDE automatically color codes predefined items in the sketch to improve its readability. These include variable types (e.g. `int` and `void`), built-in function names (e.g., `setup`, `loop`, `pinMode`, `digitalWrite`, and `delay`), and predefined constants (e.g., `HIGH`, `LOW`, and `OUTPUT`). Notice also the use of capitalization. By convention, names of variables and functions begin with lower case and capitals are used to separate words (called camel case). Names of constants are in all caps. Indentation is used to improve readability. Statements within curly brackets are indented and the terminating bracket is on a line by itself.

Name: _____

Date: _____

Part 1: Digital Output: Blinking LED

Connect the USB cord to your Arduino to turn it on. You should see a power LED on the Arduino board light up. Before you run any programs, do the following:

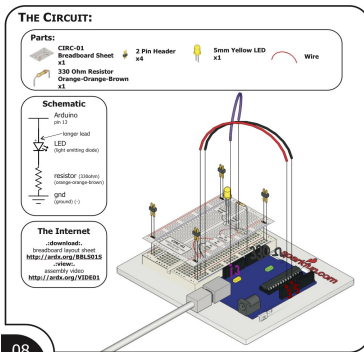
Measure the actual voltage of the two power pins. Use GND as the reference.

3.3 V Power Pin actual voltage: 3.304 [V]

5 V Power Pin actual voltage: 4.917 [V]

Build the basic circuit described in CIRC-01 (page 8) of the **Sparkfun Inventor's Guide**, except connect the anode lead (longer lead) of the LED to the Arduino's 5 V Power Pin, instead of Digital Pin 13. Does the LED light up? If not, your LED is probably connected backwards.

How much current does the LED draw? You can measure the current by measuring the voltage drop across the 330 Ω resistor in series with the LED.



330 Ω
Voltage drop across 330 Ω resistor: 2.80 [V]

Current drawn by LED: 4.1 [A]

Next, configure the circuit exactly as shown in CIRC-01 by connecting the anode lead of the LED to the Arduino's Digital Pin 13. This pin can source a maximum of 40 mA. Is that enough to power the LED?

✓ Yes No

The blinking LED sketch demonstrates how you can use the Arduino's digital output to control an external device. Open the **Blink** sketch from the Arduino IDE menu:

File > Examples > 01.Basic > Blink

To upload the sketch to the Arduino click on the right-arrow button at the top of the IDE.



Describe the result:

The LED turns on and off periodically

CODE (no need to type everything in just click)

CIRC-01

File > Examples > Digital > Blink
(example from the great arduino.cc site, check it out for other ideas)

```
/* Blink
 * Turns on an LED on for one second, then off for one second,
 * repeatedly.
 * Created 1 June 2005 by David Cuatrecasas
 * http://arduino.cc/en/tutorial/blink
 * based on an original by H. Barragan for the wiring i/o board
 */

int ledPin = 13; // LED connected to digital pin 13

// The setup() method runs once, when the sketch starts
void setup() {
  pinMode(ledPin, OUTPUT); // initialize the digital pin as an output:
}

// the loop() method runs over and over again,
// as long as the Arduino has power
void loop() {
  digitalWrite(ledPin, HIGH); // set the LED on
  delay(1000);                // wait for a second
  digitalWrite(ledPin, LOW);  // set the LED off
  delay(1000);                // wait for a second
}
```

Name: _____

Date: _____

Use an oscilloscope to observe the voltage output of Digital I/O Pin 13.

Measured how long the voltage is high: _____ [s]

Measured how long the voltage is low: _____ [s]

Discuss the agreement of these times with the delay times specified in the program?

This seems to agree perfectly with the delay times specified in the program.

Make the changes to your sketch suggested in the **Making It Better/Change the blink time** section on page 9 of the **Sparkfun Inventor's Guide**.

How fast can you make the LED blink before you can't see it flashing anymore? (Note that the period is the time of a full cycle when the LED is on and off!)

Maximum visible flashing frequency: 50 [Hz]

MAKING IT BETTER

Changing the pin:

The LED is connected to pin 13 but we can use any of the Arduino's pins. To change it take the wire plugged into pin 13 and move it to a pin of your choice (from 0-13) (you can also use analog 0-5, analog 0 is 14...)

Then in the code change the line:

```
int ledPin = 13; -> int ledPin = newpin;
```

Then upload the sketch: (ctrl-u)

Change the blink time:

Unhappy with one second on one second off?

In the code change the lines:

```
digitalWrite(ledPin, HIGH);  
delay(time on); //(seconds * 1000)  
digitalWrite(ledPin, LOW);  
delay(time off); //(seconds * 1000)
```

Control the brightness:

Along with digital (on/off) control the Arduino can control some pins in an analog (brightness) fashion. (more details on this in later circuits). To play around with it.

Change the LED to pin 9: (also change the wire)

```
ledPin = 13; -> int ledPin = 9;
```

Replace the code inside the { }'s of loop() with this:

```
analogWrite(ledPin, new number);
```

(new number) = any number between 0 and 255.

0 = off, 255 = on, in between = different brightness

Fading:

We will use another included example program. To open go to

File > Examples > Analog > Fading

Then upload to your board and watch as the LED fades in and then out.

Why is this important and how it relates the power mains frequency in the US (60 Hz) and Europe (50Hz)?

It seems like this frequency is directly related to the 'refresh rate' or maximum refresh rate of the human eye

Name: _____

Date: _____

Part 2: Digital Input: Button

Modify the circuit used in Part 1 by connecting one end of a jumper wire to the Arduino's Digital Pin 2. Leave the other end of the jumper wire unconnected.

Open the **Button** sketch from the Arduino IDE menu.

File > Examples > 02.Digital > Button

Upload the sketch to the Arduino and watch the LED while you move your hand around in the vicinity of the Arduino. What happens?

The LED activates when our hand gets close to the wire

Now connect the free end of the jumper wire to the Arduino's 5 V Power Pin. What happens to the LED as you move your hand around in the vicinity of the Arduino?

It remains always on

Finally, connect the free end of the jumper wire to the Arduino's GND. What happens to the LED as you move your hand around in the vicinity of the Arduino?

It never turns on

You have just learned the important lesson: A floating digital input will have an unpredictable value. If you want it to be high, you must connect it to 5 V. If you want it to be low, you must connect it to 0 V. This is done by using pull-up and pull-down resistors, as shown in Figure 3.

Pull-up and pull-down resistors are often used with switches to ensure that the state of the digital input is determined whether the switch is open or closed. The value typically used for pull-up and pull-down resistors is 1~10 k Ω .

Name: _____

Date: _____

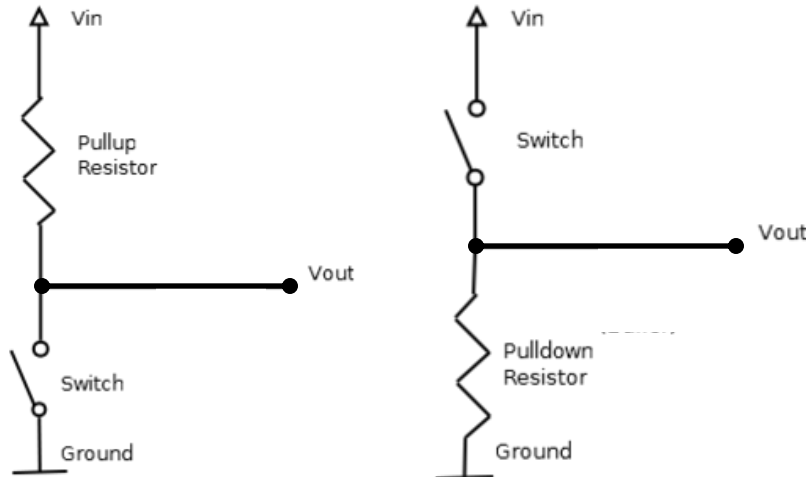


Figure 3. Pull-up resistor (left) and pull-down resistor (right).

Explain how the pull-up resistor works (i.e., What is V_{out} when the switch is open and what is it when the switch is closed? Why does V_{out} have these values?). Note that the logic state of V_{out} is defined in terms of voltage, not current. You can assume that the Arduino's Digital I/O Pins have a very high input impedance and that $I_{out} = 0$.

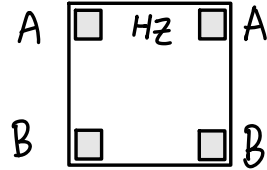
Pull up: $V_{out} = V_{in}$ when open
 $V_{out} = 0V$ when closed

Explain how the pull-down resistor works (i.e., What is V_{out} when the switch is open and what is it when the switch is closed? Why does V_{out} have these values?)

Name: _____

Date: _____

The circuit you are going to construct next will use one of the large buttons in the Inventor's Kit. This button has four leads. Use your DMM to identify the pairs of leads that are switched when the button is pressed. Draw the button below and identify one pair of leads as AA and the other as BB.



The switch is _____ NO (Normally Open) ☒ NC (Normally Closed)

Now construct the circuit described in CIRC-07 (page 20) of the **Sparkfun Inventor's Guide**, except do not add the second button that is connected to the Arduino's Digital Pin 3. You will see that this circuit uses a 10 kΩ pull-up resistor.

The LED is normally ☒ On _____ Off

When you press the button the LED goes _____ On ☒ Off

Modify the sketch to invert the behavior you witnessed above. Write the line(s) of the code you had to change below (before and after the change)

THE CIRCUIT:

Parts:
 CIRC-07 Breadboard Sheet x1
 10k Ohm Resistor Brown-Black-Orange x2
 2 Pin Header x4
 330 Ohm Resistor Orange-Orange-Brown x1
 Pushbutton x2
 Wire
 Red LED x1

Schematic

The Internet
 .download:
 breadboard layout sheet
<http://ardx.org/BBS075>
 .view:
 assembly video
<http://ardx.org/VIDE07>

CIRC-07

CODE (no need to type everything in just click)
File > Examples > Digital > Button
 (example from the great arduino.cc site check it out for other great ideas)

```

/* Button
 * by DojoDave <http://www.0j0.org>
 * Turns on and off a light emitting diode(LED) connected to digital
 * pin 13, when pressing a pushbutton attached to pin 7.
 * http://www.arduino.cc/en/Tutorial/Button
 */
int ledPin = 13;           // choose the pin for the LED
int inputPin = 2;          // choose the input pin (for a pushbutton)
int val = 0;               // variable for reading the pin status

void setup() {
  pinMode(ledPin, OUTPUT); // declare LED as output
  pinMode(inputPin, INPUT); // declare pushbutton as input
}

void loop(){
  val = digitalRead(inputPin); // read input value
  if (val == HIGH) {           // check if the input is HIGH
    digitalWrite(ledPin, LOW); // turn LED OFF
  } else {
    digitalWrite(ledPin, HIGH); // turn LED ON
  }
}

```

switch if/else logic and now
LED stays off and turns on
when switch pressed

Name: _____

Date: _____

Part 3: Analog Input: Reading voltage from a potentiometer

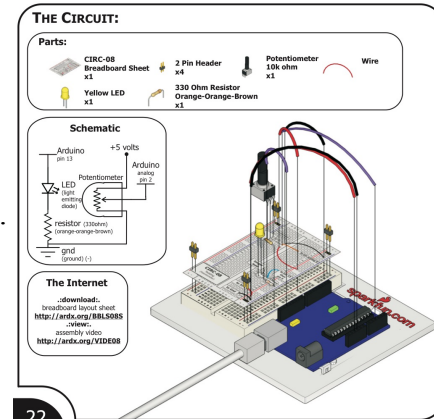
This experiment demonstrates how the Arduino can read an analog voltage and use this information to control an external device. Build the circuit described in CIRC-08 (page 22) of the **Sparkfun Inventor's Guide**. Connect the middle pin of the potentiometer to Analog pin 0 (A0) and not to digital pin 2!

Open and upload the **AnalogInput** sketch from the Arduino IDE menu.

File > Examples > 03.Analog > AnalogInput

Describe what happens to the LED when you turn the potentiometer knob.

Varying the potentiometer modulates the refresh rate of the LED



Modify the sketch to “print” values from the Arduino back to your computer. Add the following line of code anywhere in the `setup()` function

```
Serial.begin(9600);
```

Add the following line after `sensorValue = analogRead(sensorPin);` statement in the `loop()` function

```
Serial.println(sensorValue);
```

After you upload the modified sketch to the Arduino, click on the magnifying glass icon in the upper right corner of the Arduino IDE to open the serial monitor.



Use your voltmeter to read the voltage output of the potentiometer middle pin and complete the table below.

Potentiometer	DMM Voltage [V]	<code>analogRead()</code> Value
Min	0.002	0
Max	4.92	1023
Smallest Change	0.00479	$\Delta = 1$

You may not be able to measure the voltage difference to complete the last line of the table, but you can infer this from the minimum and maximum values measured.

Name: _____

Date: _____

As you (hopefully) reported in the table above, the default setting of the Arduino is to measure analog voltages in the range from 0 to +5 V. The input voltage is digitized by the microcontroller using a 10-bit ADC (Analog to Digital Converter). The decimal equivalent of this digital value is returned by the `analogRead()` function.

How many different voltage levels can be measured using 10 bits?

Number of voltage levels using 10 bits: 1024

What is the resolution of the Arduino's default analog voltage measurement?

Default analog resolution: 5 [V]

Complete the linear mathematical equation below

Input Voltage on pin A0 = 0.48mV · `analogRead()`

Sometime you might need to measure analog voltages over a smaller range, say from zero to +1 V. With the default setting of zero to +5 V you would be only using 1/5th of the Arduino's digitizing capacity. The Arduino has a built-in way of dealing with this.

The UNO's analog input pins actually measure voltages in the range zero to AREF, where AREF is the last pin on the digital I/O side of the board. If the max voltage you are going to measure is less than 5 V, you can decrease AREF, and thus increase the resolution of your measurement. AREF can be made any value between 1.1 V and 5 V by using a voltage divider; however, you need to know what you are doing or you could damage the Arduino. A safer approach is to use the two internal values of AREF that are already built into the UNO: +1.1 V or +5 V. This can be done by adding the `analogReference()` statement to the `setup()` function:

```
analogReference(INTERNAL); // sets AREF to +1.1 V
analogReference(DEFAULT); // sets AREF to +5 V
```

The default statement is only needed if you have changed AREF and want to get back to the default value.

Adjust the potentiometer until the DMM reads +0.5 V. Run the **AnalogInput** sketch and record the value returned by `analogRead()` when AREF = 5V.

(AREF = +5 V) For +0.5 V `analogRead()` = 101

Now change AREF to +1.1 V and run **AnalogInput** again

(AREF = +1.1 V) For +0.5 V `analogRead()` = 404

Which value of AREF gives you better resolution? 5 V ✓ or 1.1 V _____

Name: _____

Date: _____

Part 4: “Analog” Output: Controlling LED brightness with PWM

We will modify the circuit constructed in Part 3 to vary the brightness of the LED using the Arduino’s “analog” output. The word *analog* is in quotation marks because the Arduino cannot generate a true analog output (a voltage other than 0 or 5V). Instead it uses PWM as an approximation (see the introduction on page 3).

Move the anode lead of the LED from Digital I/O Pin 13 to Digital I/O Pin 9. This will allow you to use PWM output (note ~ symbol in front of the 9 on Digital I/O Pin 9).

Open a new sketch in the Arduino IDE and copy and paste the code below into it.

```
int sensorPin = A0;    // select the input pin for the potentiometer
int ledPin = 9;        // select the pin for the LED
int sensorValue = 0;   // variable to store the value coming from the sensor
int ledValue = 0;      // variable to store the value for the LED

void setup() {
  // declare the ledPin as an OUTPUT:
  pinMode(ledPin, OUTPUT);
}

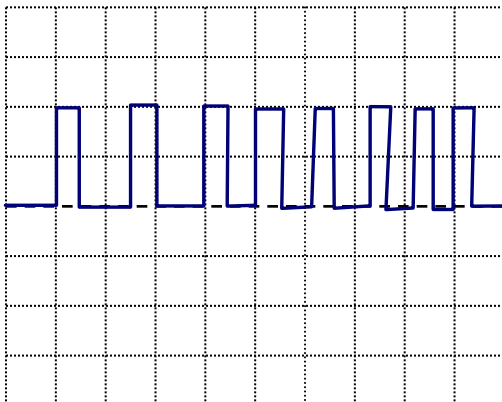
void loop() {
  // read the value from the sensor:
  sensorValue = analogRead(sensorPin);
  // map input range (0, 1023) to output range (0, 255)
  ledValue = map(sensorValue, 0, 1023, 0, 255);
  // write analog value to LED
  analogWrite(ledPin, ledValue);
  // wait 2 milliseconds
  delay(2);
}
```

Upload the sketch and describe what happens when you turn the potentiometer knob.

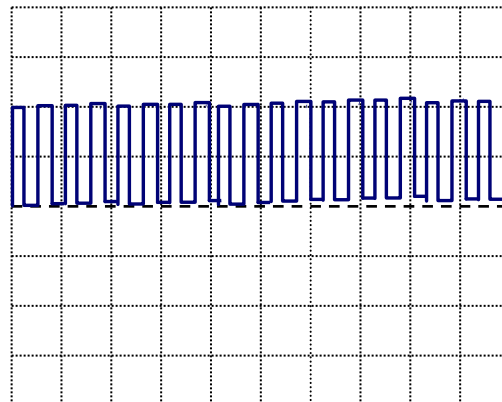
The light intensity is modulated by the knob w/o blinking off

Use an oscilloscope to observe the output voltage on Digital Pin 9 and record the waveforms below for the LED conditions indicated.

Low LED intensity



High LED intensity



Name: _____

Date: _____

Explain how the LED brightness is being controlled. What is changing in the output (Pin 9) voltage when you turn the potentiometer knob?

Adjusting the knob adjusts the width between pulses - this results in an observed change in luminosity - while the on/off frequency remains the same; the length of time between on/off cycles is modulated

What is PWM?

- pulse width modulation

What is the PWM frequency?

Measured PWM frequency 490 [Hz]

What is important about this frequency? What would happen if it were 10 Hz?

@ 10 Hz we would be able to see the light blink, 490 Hz is important because it corresponds to the refresh rate of the human eye.

Dave

Appendix: UNO Specifications

(<http://arduino.cc/en/Main/arduinoBoardUno>)

Summary

Microcontroller	ATmega328
Operating Voltage	5V
Input Voltage (recommended)	7-12V
Input Voltage (limits)	6-20V
Digital I/O Pins	14 (of which 6 provide PWM output)
Analog Input Pins	6
DC Current per I/O Pin	40 mA Source or Sink
DC Current for 3.3V Pin	50 mA Source or Sink
DC Current per VCC and GND Pins	200.0 mA
Flash Memory	32 KB (ATmega328)
SRAM	2 KB (ATmega328)
EEPROM	1 KB (ATmega328)
Clock Speed	16 MHz
Length	68.6 mm
Width	53.4 mm
Weight	25 g

Power

The Arduino Uno can be powered via the USB connection or with an external power supply. The power source is selected automatically.

External (non-USB) power (7-12V) can come either from an AC-to-DC adapter (wall-wart) or battery. The adapter can be connected by plugging a 2.1mm center-positive plug into the board's power jack. Leads from a battery can be inserted in the GND and Vin pin headers of the POWER connector.

Memory

The ATmega328 has three types of memory. A 32 KB *non-volatile* flash memory (of which 0.5 KB used for the bootloader) stores sketches. Variables used in a sketch are stored separately in a 2 KB static RAM (SRAM). Unlike the flash memory, the SRAM is *volatile* and loses its content when the board is powered down. In addition, there is a 1 KB non-volatile EEPROM (which can be read and written with the [EEPROM library](#)) to store data permanently.

Input and Output

Each of the 14 digital pins on the Uno can be used as an input or output, using [pinMode\(\)](#), [digitalWrite\(\)](#), and [digitalRead\(\)](#) functions. They operate at 5 volts. Each pin

Name: _____

Date: _____

can provide or receive a maximum of 40 mA and has an internal pull-up resistor (disconnected by default) of 20-50 kOhms.

The Uno has 6 analog inputs, labeled A0 through A5, each of which provide 10 bits of resolution (i.e. 1024 different values). By default they measure from ground to 5 V, though it is possible to change the upper end of their range using the AREF pin and the [analogReference\(\)](#) function. These pins can also be reconfigured for digital input/output using the `pinMode()` function (see above).

In addition, some pins have specialized functions:

- **UART: Pins 0 (RX) and 1 (TX)** used by the UART (Universal Asynchronous Receiver Transmitter) communication interface to receive (RX) and transmit (TX) TTL serial data. These pins are connected to the corresponding pins of the ATmega8U2 USB-to-UART chip.
- **SPI: Pins 10 (SS), 11 (MOSI), 12 (MISO), 13 (SCK)** support SPI (Serial Peripheral Interface) communication using the [SPI library](#).
- **TWI or I2C: Pins A4 (SDA) and A5 (SCL)** support TWI (Two Wire Interface, aka I2C) communication using the [Wire library](#).
- **External Interrupts: Pins 2 and 3.** These pins can be configured to trigger an interrupt on a low value, a rising or falling edge, or a change in value. See the [attachInterrupt\(\)](#) function for details.
- **PWM: Pins 3, 5, 6, 9, 10, and 11.** Provide 8-bit PWM (pulse width modulation) output with the [analogWrite\(\)](#) function.
- **LED: Pin 13.** There is a built-in LED connected to digital pin 13. When the pin is HIGH value, the LED is on, when the pin is LOW, it's off.

There are a couple of other pins on the board:

- **AREF.** Reference voltage for the analog inputs. Used with [analogReference\(\)](#).
- **Reset.** Bring this line LOW to reset the microcontroller. Typically used to add a reset button to shields which block the one on the board.